Book A Doctor

Doctor Appointment Booking App

1. Introduction:-

Project Title: Doctor Appointment Booking System

Team Members:

- GAJULA ASHOK – Project Setup & Backend.
- Chandrika Reddy Dharmavaram–Project Architecture & Frontend Dev.
- C Harshitha – Database Dev & Config and Project Execution

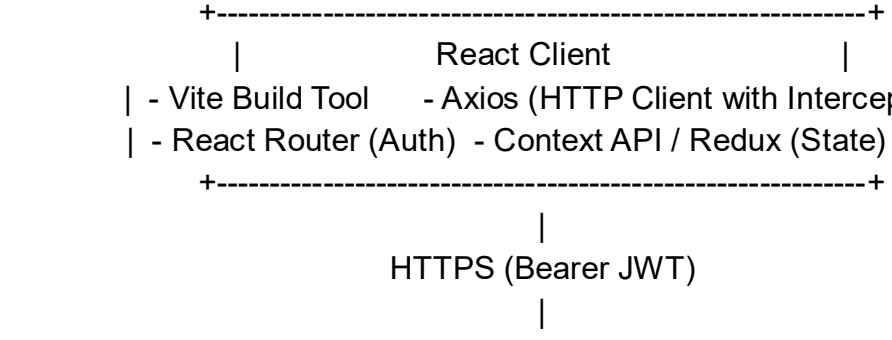
2. Project Overview:-

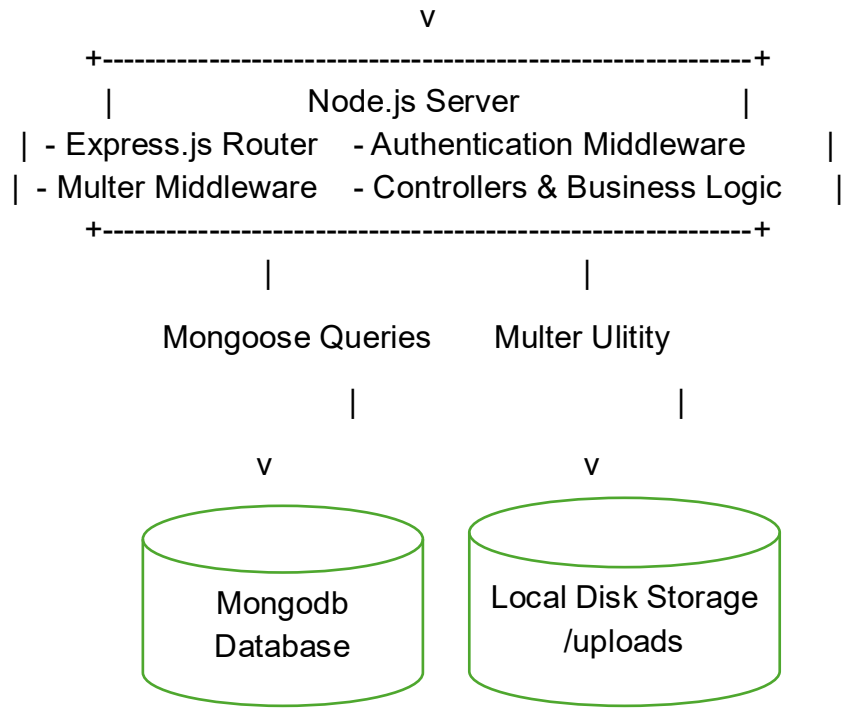
Purpose: MediCare is a modern, responsive, production-ready MERN Stack healthcare platform designed to streamline doctor-patient interactions. It serves as a central hub where patients can search for certified specialists and book consultations, doctors can manage daily schedules and inspect medical reports, and administrators exercise complete control over profile approvals, user access, and platform statistics.

3. Features:-

- **Role-Based Authentication**: Secure JWT access for Patients, Doctors, and Administrators.
- **Patient Portal**: Search/filter doctors by specialty/fee, book dates, upload medical reports, cancel consultations, track notifications, and modify profiles.
- **Doctor Onboarding**: Patients apply to become practitioners by uploading credentials. Admins verify licenses in a dedicated inspection dashboard before activating doctor profiles.
- **Doctor Panel**: Visual daily booking calendar, toggle availability hours, and approve/reject/complete appointments.
- **Admin Panel**: Graphical data charts (using Chart.js) tracking monthly bookings, total revenue tracking, direct doctor onboarding, and deletion privileges for user profiles.
- **Modern UI Aesthetics**: Custom light/dark themes, glassmorphism headers, rounded card animations, and mobile responsive drawer navigation

4. Architecture:-





- **Frontend**: Single Page Application (SPA) built using React.js and compiled via Vite. Core states (theme variables, logins, unread alerts) are coordinated globally via React Context Providers (ThemeContext, AppContext). Axios is configured with interceptors to automatically attach JWT authorization headers to outgoing queries. UI renders using Bootstrap 5 and React Bootstrap elements.
- **Backend**: RESTful API engine developed in Node.js and Express.js using a modular MVC (Models-Views-Controllers) design. The routing layer is guarded by JWT verification and role authorization middlewares. Includes global error boundaries that dynamically intercept Mongoose ValidationError, duplicate keys, and bad CastErrors, returning neat JSON messages.
- **Database**: Relational schema structure mapped on a non-relational MongoDB database using Mongoose ODM. Schemas are modularized into five primary collections (users, doctors, doctor_applications, appointments, and notifications), linked together via MongoDB ObjectId references.

5. Setup Instructions:-

- **Prerequisites**:
  a) Node.js (v18.0.0 or higher)
  b) NPM (v9.0.0 or higher)
  c) MongoDB (v6.0 or higher, running locally or accessed via a MongoDB Atlas connection string)
- **Installation**:
  - Navigate to the project root directory:

        - Bash ➔ cd C:\Users\chiru\.gemini\antigravity\scratch\medicare
    - Configure Environment Variables for Backend: Create a .env file in the backend/ directory with the following variables:
        - Env ➔
            - PORT=5000
            - MONGODB_URI=mongodb://127.0.0.1:27017/medicare
            - JWT_SECRET=supersecretjwtkey12345!@#
            - NODE_ENV=development
    - Install backend dependencies:
        - Bash ➔
            - cd backend
            - npm install
    - Install frontend dependencies:
        - Bash ➔
            - cd ../frontend
            - npm install

6. Folder Structure:-

Client (React Frontend)
frontend/
├── public/
├── src/
│ ├── components/ # Reusable UI elements (Navbar, Sidebar, ProtectedRoute, Loader)
│ ├── context/ # Global state providers (AppContext, ThemeContext)
│ ├── pages/ # Routing views
│ │ ├── admin/ # Administrator dashboards (ManageUsers, ManageDoctors, Applications)
│ │ ├── doctor/ # Doctor profile and schedule screens
│ │ ├── user/ # Patient dashboard and profile editing
│ │ └── (General Pages) # Home, Login, Register, Search listing, Details, Policy terms
│ ├── services/ # API communications client (api.js)
│ ├── utils/ # Formatting helper scripts (helpers.js)
│ ├── App.jsx # Router configuration tree
│ ├── index.css # Global styling variables, themes, and badges
│ └── main.jsx # React DOM render entry point
├── index.html

├── package.json
└── vite.config.js
Server (Express Backend)
backend/
├── config/ # MongoDB connection configurations
├── controllers/ # Business logic handlers (Auth, Doctors, Bookings, Admin, Alerts)
├── middlewares/ # Token decoding, upload configs, and global error boundaries
├── models/ # Mongoose database schemas
├── routes/ # API endpoints routes definition
├── uploads/ # Local storage folder for avatars, reports, and certificates
├── .env # Environment environment keys
├── package.json
├── server.js # Express setup, seeding script, and listener startup
└── verify_apis.js # Diagnostics script checking database connection

---

7. Running the Application

Provide commands to start the frontend and backend servers locally:
Backend: Open a terminal inside the backend/ directory and run:
Bash
npm run dev
Frontend: Open a separate terminal inside the frontend/ directory and run:
Bash
npm run dev*)*

8. API Documentation

7.1 Authentication Endpoints

- POST /api/auth/register:
  Description: Registers a new patient account with profile photo file attachment.
  Auth Required: No (Public)
  Headers: Content-Type: multipart/form-data
  Request Body: Form fields
  containing name, email, password, phone, age, gender, address, and file field profilePhoto (optional).

7.2 Doctor & Onboarding Endpoints

- GET /api/doctors:
  Description: Returns all approved, active doctor profiles. Supports search queries and pagination filters.

Auth Required: No (Public)
Parameters: search (name, specialty, hospital), specialization, hospital, experience, maxFee, availability (day of week), page, limit.

- POST /api/doctors/apply:
  Description: Submits an application for a patient to become a verified doctor.
  Auth Required: Yes (Patient)
  Headers: Content-Type: multipart/form-data

9. Authentication:-

- **JWT (JSON Web Tokens)**: We use stateless token-based authorization. When users submit valid email-password credentials via /api/auth/login, our backend generates a signature token using jsonwebtoken holding the user's ObjectId as a payload. This signature expires in 30 days.
- **Sessions/Local Storage**: The client intercepts the login response and saves the token (medicare-token) and user object (medicare-user) to localStorage. On subsequent render cycles, Axios interceptors read the key and automatically append Authorization: Bearer <token> to request headers.
- **Role Guards**: API requests are filtered through middlewares. The protect middleware decodes tokens and injects the user model into req.user. The restrictTo(...roles) middleware verifies if the user's role contains permission to access the target route. If unauthorized, it returns a 403 Forbidden response.

10. User Interface:-

- The interface incorporates a premium medical aesthetic:
- **Color Palette**: Royal Blue (#2563eb), Turquoise Accent (#06b6d4), Slate Background (#f8fafc), and White rounded cards.
- **Light / Dark Mode**: Custom CSS tokens dynamically swap based on the document's data-theme attribute (persisted in local storage).
- **Sidebar Layout**: A collapsible sidebar tailored specifically for each role coordinates dashboard sections without full-page reloads.
- **Animations**: All primary pages use @keyframes fadeIn transitions to slide up and appear smoothly.

11. Testing:-

- **Programmatic Test Diagnostics**: We supply a diagnostic script verify_apis.js in the backend/ folder. Executing node verify_apis.js checks:
  a. Environment files (.env) for validation keys.
  b. Correct module loads for all Mongoose models.
  c. Database ping connections to MongoDB.

- **Manual Verification Workflows**:
    a. **Onboarding Test**: Patients register -> Fill "Apply as Doctor" uploading certificates -> Admin logs in -> Approves application -> Doctor profile is created -> User role is converted to "doctor".
    b. **Booking Test**: Patient logs in -> Searches and picks the approved doctor -> Submits booking -> Doctor logs in -> Accepts or completes consultation.

12.Screenshots or Demo:-

Live Demo Url: http://localhost:5173 (Local deployment server

13. Known Issues:-

- **Ephemeral Files Upload**: Uploaded avatar pictures, certificates, and medical reports are stored directly on the local server disk (backend/uploads/). Deploying the project to ephemeral cloud servers (such as Heroku or Render without persistent volumes) will cause uploaded files to be deleted during container restarts.
    a. *Solution for Production*: Configure a cloud file store provider (like Amazon S3 or Cloudinary) and update uploadMiddleware.js to upload buffers directly to cloud containers.

14. Future Enhancements:

- **Telehealth Consultations**: Integrated WebRTC video room conferencing directly within doctor/patient dashboard portals.
- **Payment Processing**: Stripe or PayPal checkout flow processing patient consult fees upon booking confirmations.
- **SMS Alerts**: Twilio notification pipeline alerts patients via text messages when appointments are accepted or completed.

Thank You

PRESENTED BY:

GAJULA ASHOK

C Harshitha

Chandrika Reddy Dharmavaram